

Food Store — TP5

Índices, vistas y vistas materializadas (asistido por IA)

Base de Datos II — UTN · Equipo G · 22/09/2026

Plan de indexado con EXPLAIN antes/después (Parte A) · costo de escritura y descartes

3 vistas de reportes + vista de seguridad con verificación EXCEPT 0/0 (Parte B)

Vista materializada ×41.000 con índice único para REFRESH CONCURRENTLY (Parte C)

Flujo Kiro → OpenCode → cifras del motor → commits por pieza · DUIA

FOOD STORE — TP5: informe de mediciones

Base de Datos II — UTN · Equipo G · 22/09/2026 Base de trabajo: copia **food_store_tp5** (clonada de food_store_tp3 vía `createdb -T`; 200.003 pedidos, 500.014 líneas, 50.006 productos, 20.003 clientes, respaldo previo en `respaldos/food_store_tp3_20260922_antes_tp5.sql`). Servidor en caliente, PostgreSQL 18. Mediciones: `EXPLAIN (ANALYZE, BUFFERS)`, 3 corridas, se reporta la mejor. Los planes textuales completos están en `docs/planes5/`.

Flujo: spec → Kiro → `specs/` · generación → OpenCode → `sql/*.sql` · verificación → motor (EXCEPT / EXPLAIN) · versionado → commits por pieza.

Parte A — Plan de indexado asistido por IA

A.1 Consultas frecuentes y planes antes/después

Consulta (frecuencia)	Nodo dominante ANTES	Nodo dominante DESPUÉS	Tiempo ANTES	Tiempo DESPUÉS	Mejora
A1 pedidos del mes (<code>pedido</code> , mensual)	Seq Scan 200k + filtro + Sort 1556 kB	Bitmap Index Scan <code>ix_pedido_fecha_estado</code> (16.809) + Sort	18.8 ms	11.5 ms	×1.6
A2 productos a reponer (<code>producto</code> , diario)	Seq Scan 50k + filtro + Sort	Index Scan <code>ix_producto_stock_reposicion</code> (parcial) + Incremental Sort presorted (226 filas→100)	10.4 ms	0.34 ms	×30.4
A3 cliente por apellido (<code>cliente</code> , decenas/día)	Seq Scan 20k + Sort	Bitmap Index Scan <code>ix_cliente_apellido_nombre</code> (1.111) + top-N heap sort	2.75 ms	1.40 ms	×2.0

Consultas en `sql/queries_tp5.sql`. Planes en `docs/planes5/{A1_reporte_mensual,A2_stock_reposicion,A3_cliente_apellido}_{antes,despues}.txt`.

A.2 Índices aceptados (`sql/indices.sql`)

- `ix_pedido_fecha_estado` (`fecha`, `estado`) — compuesto; `fecha` líder (selectiva, ~8 % del mes) convierte el rango en entrada de árbol y sirve a la variante `estado = 'ENTREGADO'` (dashboard mensual). Build: 439 ms.
- `ix_producto_stock_reposicion` (`stock`) `WHERE stock < 50` — **índice parcial**: indexa solo las 12,4 k filas que interesan (de 50 k): menor espacio y mantenimiento. Build: 36 ms. El `Incremental Sort` presorted lee solo 226 filas para entregar 100.
- `ix_cliente_apellido_nombre` (`apellido text_pattern_ops`, `nombre text_pattern_ops`) — clase de operador `text_pattern_ops` para explotar `LIKE 'prefijo%'` con B-tree (collations no-C); compuesto presorted para la paginación. Build: 50 ms.

A.3 Índice descartado (sobreindexación) — D1

Propuesta de la IA: `CREATE INDEX idx_detalle_cantidad ON detalle_pedido (cantidad)`.

Se comprobó **empíricamente** en transacción reversible (docs/planes5/D1_q3_top_unidades_con_indice.txt): con el índice creado, el plan de Q3 (top 10 por unidades) **sigue siendo** `Seq Scan` **sobre las 512 k líneas** + HashAggregate (316 ms). Es una agregación de tabla completa: el índice jamás aparece y solo pagaría mantenimiento en cada INSERT. ****Descartado**.

A.4 Índice descartado (redundante) — D2

Propuesta: `idx_pedido_cliente_estado (cliente_id, estado)`. Redundante con `idx_pedido_cliente (cliente_id)`: ~10 pedidos por cliente (200 k / 20 k), el filtro extra por `estado` (cardinalidad 4) no cambia el plan de Q2 y sí suma costo de escritura. **Descartado** (ver specs/descartado_pedido_cliente_estado.md).

A.5 Costo de los índices sobre la escritura

Carga de **6.000 pedidos + 6.000 líneas** (misma forma antes/después; `sql/prueba_escritura_antes.sql` y `prueba_escritura_despues.sql`):

Operación	ANTES	DESPUÉS	Δ
INSERT 6.000 × <code>pedido</code> (mantiene <code>ix_pedido_fecha_estado</code>)	234.5 ms	237.8 ms	+1.4 %
INSERT 6.000 × <code>detalle_pedido</code> (los índices nuevos NO están sobre esta tabla)	416.8 ms	304.7 ms	dentro del ruido individual

Lectura honesta: el único índice nuevo que se paga en cada escritura de la prueba es el compuesto de `pedido` (+1,4 %, aceptable frente a ×1.6–30 en lectura); `detalle_pedido` no cambia porque ningún índice nuevo lo toca.

Parte B — Vistas para los reportes (`sql/views.sql`)

Vista	Exposición	Filas	vs consulta manual (EXCEPT)
<code>v_reporte_productos_vigentes</code>	producto vigente + nombre de categoría (oculta <code>created_at</code>)	50.006	0 / 0
<code>v_reporte_pedidos_cliente</code>	pedidos con datos del cliente (oculta <code>telefono</code> , <code>created_at</code>)	212.003	0 / 0
<code>v_reporte_detalle_pedido</code>	línea + nombre de producto + <code>importe</code> derivado (R4)	512.014	0 / 0
<code>v_cliente_contacto</code> (SEGURIDAD)	cliente sin <code>telefono</code> (id, nombre, apellido, email)	20.003	0 / 0

Verificación en `sql/verificacion_views.sql`: filas idénticas + `EXCEPT` en ambas direcciones = 0 para las 4 vistas, y comprobación de que `telefono` no es columna de `v_cliente_contacto` (0 columnas en `information_schema`).

Adaptación de seguridad documentada: el enunciado pide "usuario sin contraseña"; el modelo local no tiene tabla `usuario` ni columna `contraseña` (el alta es `cliente`, R6). El dato personal que oculta la vista

de seguridad es `telefono`: `GRANT SELECT ON v_cliente_contacto` sin acceso a la tabla base. Ver `specs/vista_cliente_contacto_seguridad.md`.

Parte C — Vista materializada (`sql/materializadas.sql`)

Reporte costoso elegido: **facturación por categoría, mes y forma de pago** (117 filas), que re-escanea 500 k líneas por corrida.

Método	EXPLAIN (best-of 3)	Build
Consulta original (4 tables, agregado)	905.8 ms	—
<code>mv_facturacion_categoria_mes_pago</code> (WITH DATA)	0.022 ms	908 ms

Mejora: ×41.000. La MV tiene índice **ÚNICO** `uq_mv_facturacion_cat_mes_pago (categoria, mes, forma_pago)`, requisito para `REFRESH MATERIALIZED VIEW CONCURRENTLY` (probado, funciona) y uno de soporte por `mes`. Equivalencia MV ↔ consulta original: **EXCEPT 0 / 0** (`sql/verificacion_materializada.sql`).

Frecuencia de refresco sugerida: una vez al día, en el cierre de jornada (job nocturno, p. ej. 00:10), porque el reporte es de gerencia diaria y los pedidos ENTREGADO se cierran a lo largo del día.

Qué implica para el usuario el dato "viejo": entre refrescos la MV no refleja pedidos que acaban de pasar a ENTREGADO (ni correcciones de líneas sobre las que ya se facturó). Para este reporte es aceptable: la diferencia máxima de frescura es < 24 h y el costo de refresco en vivo (908 ms de reescritura) no se justifica en horario operativo. De cambiar el uso (p. ej. un dashboard de caja en tiempo real), se subiría la frecuencia con `REFRESH CONCURRENTLY` sin bloquear lecturas.

duia.md — Declaración de uso de IA y bitácora (TP5)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store · **Fecha:** 22/09/2026 **Herramientas:** Kiro (especificaciones; los archivos quedaron en `specs/` y son el entregable de especificación) y **OpenCode** (agente de codificación en terminal, modelo *big-pickle*) para generar el SQL a partir de cada spec. *Git/GitHub** para el historial.

Regla de cátedra respetada: la IA **propone** y escribe; el **motor y los números deciden** (EXPLAIN ANALYZE / EXCEPT); el humano **lee línea por línea y justifica** cada decisión antes de versionarla.

1. Plan de indexado (Parte A)

1.1 Índice `ix_pedido_fecha_estado`

- **Spec (Kiro):** `specs/indice_pedido_fecha_estado.md` — reporte mensual de pedidos, columnas `fecha` + `estado`, criterio: pasar de Seq Scan a Index/Bitmap.
- **Propuesta de la IA (OpenCode):** `CREATE INDEX ... ON pedido (fecha, estado);`.
- **Verificación en el motor:** antes `Seq Scan` 200 k + Sort (18.8 ms) → después `Bitmap Index Scan` sobre el índice (16.809 filas, 11.5 ms, **×1.6**).
- **Decisión:** aceptado sin modificaciones. El `Sort` persiste porque el planificador eligió Bitmap (rango del 8 %) en vez de Index Scan, y se documentó honestamente.

1.2 Índice parcial `ix_producto_stock_reposicion`

- **Spec (Kiro):** `specs/indice_producto_stock_reposicion.md` — productos a reponer, columna `stock`, reporte paginado.
- **Propuesta de la IA: índice parcial** `ON producto (stock) WHERE stock < 50`.
- **Verificación:** antes `Seq Scan` 50 k + Sort (10.4 ms) → después `Index Scan` parcial + `Incremental Sort` presorted (0.34 ms, **×30.4**).
- **Decisión:** aceptado. El parcial se justificó con el criterio de teoría (indexar solo las 12,4 k filas relevantes) y se midió.

1.3 Índice `ix_cliente_apellido_nombre`

- **Spec (Kiro):** `specs/indice_cliente_apellido_nombre.md` — buscador de clientes por apellido con paginación.
- **Propuesta de la IA:** compuesto `(apellido text_pattern_ops, nombre text_pattern_ops)`.
- **Verificación:** antes `Seq Scan` 20 k (2.75 ms) → después `Bitmap` con `Index Cond` de prefijo + top-N sort (1.40 ms, **×2**). Se documentó el rol de la **opclass** (requisito para LIKE con B-tree en collation no-C), que es contenido de la teoría de "tipos de índice".

1.4 Descartado por sobreindexación (obligatorio de la bitácora) — D1

- **Spec:** propuesta de la IA sin spec (se evaluó a contraprueba): `CREATE INDEX idx_detalle_cantidad ON detalle_pedido (cantidad);` para Q3 (top por unidades).
- **Qué propuso la IA:** "indexar cantidad para acelerar el SUM".
- **Qué se verificó:** en transacción reversible se creó el índice y se midió Q3: **el plan sigue siendo Seq Scan sobre 512 k líneas + HashAggregate** (316 ms); el índice nunca aparece. Es una agregación de tabla completa.
- **Decisión: DESCARTADO** por sobreindexación (`specs/descartado_detalle_cantidad.md`). La IA proponía delegar; el plan desconfió la intuición "un índice siempre ayuda".

1.5 Descartado por redundancia — D2

- **Propuesta:** `idx_pedido_cliente_estado (cliente_id, estado)`.
- **Qué se decidió:** redundante con `idx_pedido_cliente` (~10 pedidos/cliente) + estado de cardinalidad 4; no cambia Q2 y suma mantenimiento. Descartado (`specs/descartado_pedido_cliente_estado.md`).

1.6 Costo de escritura (Parte A.5)

- Scripts `sql/prueba_escritura_antes.sql` / `despues.sql`: pedido +1.4 %, líneas dentro del ruido (los índices nuevos no tocan `detalle_pedido`). Números en `informe_mediciones.md` §A.5.

2. Vistas (Parte B)

- **Specs (Kiro):** 4 archivos en `specs/` (`vista_reporte_productos_vigentes.md`, `vista_reporte_pedidos_cliente.md`, `vista_reporte_detalle_pedido.md`, `vista_cliente_contacto_seguridad.md`), con columnas a exponer, filtros y qué ocultar.
- **Generación (OpenCode):** `sql/views.sql` (3 vistas de reportes + vista de seguridad).
- **Verificación de equivalencia (requisito mínimo de la bitácora, Parte B.3):** `sql/verificacion_views.sql` — filas idénticas y **EXCEPT 0/0 en ambas direcciones** para las 4 vistas (50.006 / 212.003 / 512.014 / 20.003 filas). Además se comprobó que la vista de seguridad **no expone la columna telefono** (0 en `information_schema.columns`).
- **Decisión:** aceptadas sin modificaciones. La vista de seguridad aplica el criterio de mínima exposición; se documentó la **adaptación** (el modelo local no tiene `contraseña`; el dato sensible es `telefono`) para la defensa oral.

3. Vista materializada (Parte C)

- **Spec (Kiro):** `specs/materializada_facturacion_categoria_mes_pago.md`.
- **Generación (OpenCode):** `sql/materializadas.sql` — `CREATE MATERIALIZED VIEW ... WITH DATA` + índice **ÚNICO** (`categoria, mes, forma_pago`) + índice por `mes`.
- **Verificación (requisito mínimo del TP, medición):** original 905.8 ms vs MV **0.022 ms (×41.000)**; `REFRESH CONCURRENTLY` probado y funcionando; equivalencia MV ↔ original **EXCEPT 0/0**

(`sql/verificacion_materializada.sql`).

- **Decisión:** aceptada. Frecuencia de refresco sugerida y efecto del dato "viejo" en `informe_mediciones.md` §Parte C.

4. Resumen de decisiones IA → humano

Pieza	Propuso IA	Se aceptó	Se modificó	Se descartó	Evidencia
ix_pedido_fecha_estado	✓	✓	—	—	planes5/A1 ×1.6
ix_producto_stock_reposicion	✓ (parcial)	✓	—	—	planes5/A2 ×30.4
ix_cliente_apellido_nombre	✓ (opclass)	✓	—	—	planes5/A3 ×2
idx_detalle_cantidad (D1)	✓	—	—	✓ sobreindexación	plan sigue Seq Scan
idx_pedido_cliente_estado (D2)	✓	—	—	✓ redundante	análisis + specs
4 vistas	✓	✓	—	—	EXCEPT 0/0
MV + índice único	✓	✓	—	—	×41.000 + CONCURRENTLY

Ningún script se ejecutó sin leerlo línea por línea y sin haberlo cargado sobre la copia `food_store_tp5` (protocolo de seguridad de la cátedra).

FOOD STORE — TP5 (Unidad 3 · Semana 5): Índices, vistas y vistas materializadas

Trabajo integrador de **Base de Datos II (UTN)**. Continúa las Semanas 1 a 4 (`schema.sql`, `seed.sql` = `data.sql`, `queries.sql`) y agrega índices, vistas y una vista materializada **sin modificar el modelo de datos**.

Estructura mínima pedida por la cátedra:

```
food-store/
├── schema.sql          (heredado, sin modificar)
├── seed.sql / data.sql (heredado; el proyecto lo llama seed.sql)
├── queries.sql         (heredado en sql/)
├── sql/indices.sql     (nuevo – Parte A)
├── sql/views.sql       (nuevo – Parte B)
├── sql/materializadas.sql (nuevo – Parte C)
├── specs/              (especificaciones de Kiro, 11 archivos)
├── duia.md             (bitácora de uso de IA)
├── informe_mediciones.md (EXPLAIN ANALYZE antes/después, lectura y escritura)
└── README.md          (este archivo)
```

Prerrequisitos

- PostgreSQL 18 local, usuario `postgres`. (`psql/pg_dump` no están en el PATH: usar `C:\Program Files\PostgreSQL\18\bin\...`)
- Base masiva: `food_store_tp3` ya existente (Semanas 3-4) o recargar con `schema.sql` + `seed.sql`.

Cómo reproducir (sobre la copia de trabajo)

```
$env:PGPASSWORD = '<contraseña>'
$psql = 'C:\Program Files\PostgreSQL\18\bin\psql.exe'

# 1. Copia de trabajo (protocolo de cátedra) + respaldo previo
# pg_dump -U postgres food_store_tp3 > respaldos/<fecha>.sql
# createdb -U postgres -T food_store_tp3 food_store_tp5

# 2. Parte A – índices y mediciones
# a) planes ANTES: python (Temp)\run_explain_tp5.py food_store_tp3 antes 3 (o EXPLAIN a mano)
# b) escritura ANTES: & $psql -d food_store_tp5 -f sql\prueba_escritura_antes.sql
# c) crear índices: & $psql -d food_store_tp5 -f sql\indices.sql
# d) planes DESPUÉS: python (Temp)\run_explain_tp5.py food_store_tp5 despues 3
# e) escritura DESPUÉS: & $psql -d food_store_tp5 -f sql\prueba_escritura_despues.sql

# 3. Parte B – vistas
& $psql -d food_store_tp5 -f sql\views.sql
& $psql -d food_store_tp5 -f sql\verificacion_views.sql # EXPECT 0/0

# 4. Parte C – vista materializada
```

```
& $psql -d food_store_tp5 -f sql\materializadas.sql
& $psql -d food_store_tp5 -f sql\verificacion_materializada.sql
```

Los runners usan psycopg2 y archivan los planes en `docs/planes5/`. Las queries medidas están en `sql/queries_tp5.sql` (A1-A3) y la prueba de escritura en `sql/prueba_escritura_{antes,despues}.sql`.

Resultados en una línea

- **Índices:** $A1 \times 1.6 \cdot A2$ (parcial) **$\times 30.4$** $\cdot A3 \times 2$ — planes en `docs/planes5/`.
- **Escritura:** pedido + 1.4 % (costo real del índice compuesto); detalle_pedido no varía (los índices nuevos no lo tocan).
- **Descartes:** `detalle_pedido(cantidad)` (el plan sigue Seq Scan: verificado) y `pedido(cliente_id, estado)` (redundante) — `specs/descartado_*.md`.
- **Vistas:** 4 vistas verificadas con EXCEPT 0/0; la de seguridad no expone `telefono`.
- **Materializada:** 905.8 ms → **0.022 ms ($\times 41.000$)**, con índice único para `REFRESH CONCURRENTLY`.

Detalle, justificaciones y planes textuales: `informe_mediciones.md`.